High Level Design Document (HLD)

RAG Customer Support Assistant

Prepared By
Sannidhya Milind Bahulekar

Internship ID
IN226066402

College / University
Ajeenkya D.Y. Patil University

Internship Organization
Innomatics Research Labs

Submission Date
24 April 2025

Document History

Sr. No.	Date	Summary of Changes	Reviewed By
1	24-Apr-26	Draft — Internship Submission	

Contents

1. System Overview 4
1.1 Problem Statement 4
1.2 Scope 4
1.3 Design Philosophy 4
2. Architecture Overview 5
2.1 High-Level Architecture 5
2.2 End-to-End Data Flow 5
3. Component Descriptions 6
3.1 Document Loader 6
3.2 Text Chunker 6
3.3 Embedding Model 6
3.4 Vector Store (ChromaDB) 6
3.5 Retriever 6
3.6 LLM Layer 6
3.7 LangGraph Workflow Engine 6
3.8 Routing Layer 7
3.9 HITL Module 7
4. Technology Choices & Justification 7
5. Scalability Considerations 8
5.1 Handling Large Documents 8
5.2 Increasing Query Load 8
5.3 Latency Concerns 8
6. Summary 9

HIGH-LEVEL DESIGN DOCUMENT

RAG-Based Customer Support Assistant

with LangGraph Workflow Orchestration & Human-in-the-Loop Escalation

1. System Overview

1.1 Problem Statement

Customer support teams in product companies face a recurring challenge: a large portion of incoming queries are repetitive, factual questions that could be answered directly from existing documentation — policy manuals, FAQs, product guides, or service catalogs. Handling these queries manually consumes agent time, increases response latency, and reduces overall support quality.

This project addresses that challenge by designing a Retrieval-Augmented Generation (RAG) system that automatically answers user queries by pulling relevant information from a PDF knowledge base. The system is designed to handle the majority of queries autonomously and gracefully escalate the rest to a human agent when confidence is low or the query is too complex.

As part of my internship at Innomatics Research Labs, I was tasked with designing this system to demonstrate practical understanding of RAG architecture and workflow orchestration

1.2 Scope

The system covers the following functional boundaries:

- Ingesting and processing one or more PDF documents into a searchable vector store
- Accepting natural language queries from users via CLI or web interface
- Retrieving semantically relevant document chunks and generating a coherent response
- Evaluating response confidence and routing accordingly
- Escalating low-confidence or complex queries to a human support agent via HITL
- Logging all interactions for future improvement

The system does not cover: real-time document updates, multi-user session management at scale, or production deployment infrastructure (these are flagged as future enhancements).

1.3 Design Philosophy

Rather than building a simple chatbot that generates answers from scratch, this system is designed around the principle of grounded generation — the LLM is only permitted to answer based on retrieved evidence. This reduces hallucination significantly, makes the system auditable (every answer can be traced to a source chunk), and makes it easier to update the knowledge base without retraining the model. This philosophy was particularly reinforced during my internship, where accuracy and traceability were emphasized over raw generation speed.

2. Architecture Overview

2.1 High-Level Architecture

The system is organized into four distinct functional layers that operate in sequence. The first two layers (document ingestion and query processing) handle data transformation, while the last two (workflow orchestration and human escalation) handle decision-making and response delivery.

Layer	Responsibility
Document Ingestion Layer	Load PDFs, chunk text, generate embeddings, store in ChromaDB
Query Processing Layer	Embed user query, retrieve top-k chunks, generate answer via LLM
Workflow Orchestration (LangGraph)	Control node-to-node flow, apply routing logic, manage state
HITL Escalation Layer	Trigger human review, receive agent response, merge back into flow

2.2 End-to-End Data Flow

The complete lifecycle of a query moves through the following stages:

1. A PDF document is loaded from disk and split into overlapping text chunks of approximately 512 tokens each.
2. Each chunk is converted into a dense vector embedding using a sentence transformer model and stored in ChromaDB with its source metadata.
3. A user submits a natural language query through the interface (CLI or web form).
4. The query is embedded using the same model and a cosine similarity search is run against ChromaDB to retrieve the top-k most relevant chunks.
5. The retrieved chunks and original query are composed into a structured prompt and sent to the LLM for answer generation.
6. The LLM response is evaluated for confidence. If the score meets the threshold, the answer is returned to the user. If not, the query is escalated.
7. In the HITL path, a support ticket is created and the human agent reviews the query and retrieved context, then provides a response.
8. The human response is merged into the system's output, logged for feedback, and delivered to the user.

3. Component Descriptions

3.1 Document Loader

The document loader is responsible for reading the source PDF from disk and extracting its raw text content in a structured form. It preserves basic formatting signals (like paragraph breaks and heading-level text) that help the chunking module make better split decisions. In the prototype, PyMuPDF (fitz) is used because it handles complex PDF layouts more reliably than text-only extractors. The output is a list of page-level text strings passed to the chunker.

3.2 Text Chunker

The chunker breaks the extracted document text into segments small enough for an embedding model to process, while retaining enough context for the LLM to generate a meaningful answer. A recursive character-based splitter is used with a target chunk size of 512 tokens and a 50-token overlap between adjacent chunks. The overlap ensures that information at chunk boundaries is not lost during retrieval. Each chunk is stored alongside its source document name and page number.

3.3 Embedding Model

Embeddings are dense vector representations of text that capture semantic meaning. Two embedding options are considered in this design. For cost-sensitive or offline deployments, the all-MiniLM-L6-v2 model from Sentence Transformers is used — it runs locally, produces 384-dimensional vectors, and performs well on short factual passages. For higher accuracy, OpenAI's text-embedding-3-small model (1536 dimensions) can be substituted via an API call. The same model must be used for both ingestion and query embedding to ensure vector space consistency.

3.4 Vector Store (ChromaDB)

ChromaDB is an open-source, embedded vector database that requires no external server process. It persists embeddings and metadata to disk and provides a simple Python API for similarity search. The top-k search operation returns the most semantically similar chunks to a given query vector along with their distance scores, which are used as the basis for confidence estimation. ChromaDB was chosen over alternatives like Pinecone or Weaviate primarily for its zero-infrastructure requirement, which is appropriate for a prototype system.

3.5 Retriever

The retriever sits between the vector store and the LLM. It takes the embedded query vector, queries ChromaDB for the top-k most similar chunks (typically k=3 to k=5), and returns the text content and metadata of those chunks. The retriever also computes a normalized similarity score for each chunk; the average or minimum of these scores forms the basis of the confidence evaluation that drives the routing decision.

3.6 LLM Layer

The LLM receives a structured prompt composed of the user query and the retrieved chunks. The prompt explicitly instructs the model to answer only from the provided context and to indicate when the context is insufficient. This grounding constraint is critical for accuracy — without it, the model may generate plausible-sounding but unsupported answers. Claude (via the Anthropic API) or GPT-4 (via OpenAI) can be used interchangeably in this design.

3.7 LangGraph Workflow Engine

LangGraph is a library built on top of LangChain that enables the construction of stateful, multi-step AI workflows as directed graphs. Each node in the graph represents a processing step, and the edges between nodes define the flow of execution. Conditional edges allow the graph to

branch based on state values — in this system, the routing decision node reads the confidence score from the state object and directs execution to either the output node or the HITL escalation node. This approach makes the control logic explicit, testable, and easy to modify.

3.8 Routing Layer

The routing layer is a conditional edge function within the LangGraph graph. It reads the confidence score from the current graph state and applies a threshold rule: if the score is above 0.75, the answer is considered reliable and the flow proceeds to the output node. If the score is below 0.75, or if the query is detected as belonging to a sensitive intent category (account management, billing disputes, legal matters), the flow is redirected to the HITL escalation node.

3.9 HITL Module

The Human-in-the-Loop module handles the escalation path. When triggered, it creates a support ticket containing the original query, the retrieved chunks, and the LLM's attempted answer (if any). The ticket is sent to a support agent queue. The graph execution is suspended at this point using LangGraph's interrupt mechanism, and the user is notified with a ticket reference number. When the agent resolves the ticket, the graph resumes with the human response injected into the state, which is then delivered to the user and logged for training purposes.

4. Technology Choices & Justification

Component	Choice	Justification
PDF Loader	PyMuPDF (fitz)	Handles complex layouts; preserves text structure better than pdfplumber for multi-column docs
Chunker	LangChain RecursiveCharacterTextSplitter	Respects natural text boundaries; configurable overlap; well-tested in RAG pipelines
Embeddings	all-MiniLM-L6-v2 / OpenAI	MiniLM is free and fast; OpenAI offers higher accuracy at additional cost
Vector Store	ChromaDB	Zero-infra, embedded, Python-native; ideal for prototyping and local testing
LLM	Claude 3 Sonnet / GPT-4	Strong instruction-following for grounded generation; API-accessible
Workflow	LangGraph	Stateful graph execution with conditional routing and interrupt support for HITL
Interface	CLI (prototype)	Simplest path for demonstration; can be swapped for FastAPI + React frontend

5. Scalability Considerations

5.1 Handling Large Documents

For documents exceeding 200 pages, the ingestion pipeline should be parallelized. Each page's chunks can be embedded independently and batch-upserted into ChromaDB. Streaming ingestion avoids loading the entire document into memory at once. In production, this pipeline would run as an asynchronous background job triggered on document upload.

5.2 Increasing Query Load

ChromaDB in its default embedded mode is not designed for high-concurrency access. Under increased load, the vector store should be migrated to a server mode deployment (ChromaDB supports a client-server architecture) or replaced with a distributed store like Weaviate or Qdrant. The LLM API calls are the primary latency bottleneck — response caching for frequently asked questions would reduce both cost and latency significantly.

5.3 Latency Concerns

The end-to-end latency for a single query in the prototype is approximately 2 to 4 seconds, dominated by the LLM generation step. Several optimizations are available: streaming LLM responses to the user (reduces perceived latency), reducing chunk count from k=5 to k=3 (fewer tokens in the prompt), and pre-computing embeddings for common queries. The HITL path has inherently higher latency due to human response time; this is mitigated by providing the user an immediate acknowledgement with a ticket reference.

6. Summary

This high-level design describes a RAG-based customer support system that combines semantic document retrieval with LLM-powered answer generation and a human escalation path for low-confidence cases. The architecture is modular — each component can be swapped or upgraded independently — and the use of LangGraph for workflow orchestration makes the routing logic explicit and auditable. The design prioritizes correctness and grounding over raw generation speed, which is the appropriate trade-off for a customer-facing support context.